

Data Leakage in Machine Learning

Odai Nassar

Department of Industrial and Systems Engineering, Texas A&M University, Texas, USA

Abstract

Data leakage is a critical issue in machine learning that can lead to deceptively high model performance during development, followed by poor generalization to real-world data. It occurs when information from the validation or test set is inadvertently used in training, resulting in biased models that lack predictive reliability. This report explores the concept of data leakage, categorizes its common forms—such as target leakage, group leakage, time series leakage, train-test contamination, and data snooping—and illustrates each with practical examples from existing literature, including the CheXNet study by Rajpurkar et al. The consequences of data leakage can be significant, especially in high-stakes applications like healthcare or fraud detection, where misleading model accuracy may result in costly errors. To address these challenges, the report presents best practices to prevent leakage, including time-based data splitting, pipeline frameworks, and rolled-out cross-validation. By adopting these techniques and gaining a deep understanding of the underlying data, machine learning practitioners can build models that are both robust and trustworthy.

1. Introduction

A common problem in machine learning is the problem of data leakage. Data leakage is a problem where some data from the validation dataset is used unintentionally and mistakenly in developing the model. This mistake of training the model on data that should not be known at the time of developing the model leads to over-optimistic models (too good to be true) [1]. That data should not be used in training the model because that data is what the model is intended to predict. For example, when some data overlaps between the training and the validation datasets, as in the study of the research group headed by Andrew Ng that aimed at detecting pneumonia from chest X-rays [2].

Data leakage in machine learning is a serious problem that leads to unreliable and biased models. That is because the developed model was accidentally partially trained on the validation dataset, which leads to models that show unusually high performance before being launched. However, after the model is launched to process real data, a considerable difference in performance will be observed. In some cases, that may lead to costly consequences due to high error rates [3].

In this report, the problem of data leakage in machine learning is discussed along with the common types of data leakage and some of the ways to address and avoid them.

2. Common types of data leakage in machine learning

There are multiple types of data leakage in machine learning. Some of them might be obvious, but some of them require a deep understanding of the data and experience in

machine learning to avoid them. Below are some common types of data leakage in machine learning:

2.1.Target leakage:

This type of data leakage happens when information about the target variable is leaked to a feature that is used to develop the model where that feature was calculated after the target was already known. For example, suppose a model to be developed to detect fraudulent credit card transactions and the outcome of the investigation is used in building the model. In that case, this will be an example of target leakage. That is because the outcome of the investigation is inferred from the target variable and used in training the model, where it should not be available at the time of building the model. Features of this kind cannot be used in developing the model and must be removed [3].

2.2.Group leakage:

This type of data leakage occurs when the training data contains information about groups from the validation data. For example, in the paper of Andrew Ng, the data were groups of X-ray images for each patient. When the dataset was randomly split, that led to having patients that have images in both the training and validation datasets. Because of that overlapping, the model was biased and unreliable since it was trained on data that are in the validation dataset [2].

2.3.Leakage in time series:

When the data is sequential based on time (temporal data) and then randomly partitioned into a training and a validation data set, that is when the leakage in time series occurs.

2.4.Train-test contamination:

It occurs when imputing the data before splitting it into a training dataset and a validation dataset. That leads to imputing based on data that is both in the training and test datasets. When this happens, the training data set will contain information and statistics that are affected by the validation dataset and will be used in developing the model [3].

2.5.Data snooping:

This occurs when the model is finetuned and its features are screened based on the test data which leads to a biased and overfitted model that does not perform well on new datasets.

3. Ways to avoid data leakage:

It is important to be able to detect and address data leakage in your developed model to remove them and correct the model. Look for signs of unusually good performance (too good to be true). Additionally, follow a thoughtful plan that involves recommended steps to develop the model. Below are some recommended ways to avoid data leakage in machine learning.

3.1. Have a time cutoff:

Do not use features that are generated after the moment of prediction. The features that are generated after the moment of prediction are inferred from the target variable, which leads to target leakage.

3.2. Deeply understand the variables used in the model:

When you have a deep understanding of the datasets that you have, where they came from, how they were collected, how they were grouped and split, etc., you can see any potential data leakage and avoid it more easily [3].

3.3. Rolled-out cross-validation:

This approach is helpful in time series data. In this approach, a sequential part of the data is used for training the model and the last point is left out for validation. The process is repeated multiple times and the test mean squares of the error (test MSE) are averaged at the end.

3.4. Time-based splitting:

This approach is necessary for time series data in which the data must be split based on its time sequence and never randomly. If the data is randomly split, then the model might use future data to be trained and then make predictions on data in the past [3].

3.5. Split before imputation:

To avoid train-test contamination, the dataset needs to be split into a training dataset and a validation dataset before any imputation or preprocessing takes place. In general, the data needs to be split as soon as possible.

3.6. Use pipeline framework:

This is another way to avoid train-test contamination. The pipeline is set up first to include all the preprocessing steps needed as imputing, normalizing, grouping, etc. After that, the data is split into training and test data. The training data is passed through the pipeline to be processed, then the test data is passed through the same pipeline separately.

4. References:

- [1] J. Brownlee, "Data Leakage in Machine Learning," MachineLearningMastery.com. Accessed: Mar. 15, 2023. [Online]. Available: <https://machinelearningmastery.com/data-leakage-machine-learning/>
- [2] P. Rajpurkar *et al.*, "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning." arXiv, Dec. 25, 2017. Accessed: Mar. 15, 2023. [Online]. Available: <http://arxiv.org/abs/1711.05225>
- [3] "How to identify and treat data leakage," Encora. Accessed: Mar. 15, 2023. [Online]. Available: <https://www.encora.com/insights/how-to-identify-and-treat-data-leakage>